

## Unit 3 - Relational Database Design (CO3)

### Topics:

Design process steps for Relational Schema, Codd's Rules, Database Relationships and Constraints - Primary Key, Foreign Key, Null, Not Null, Unique

**Design Guidelines for Relational Schema:** Database design anomalies, Functional Dependencies, Normalization process - understanding Normal Forms based on Primary keys and converting database tables to 1NF, 2NF and 3NF, General Definitions with one example for Boyce-Codd Normal Forms (BCNF), Overview of 4<sup>th</sup> and 5<sup>th</sup> normal forms

# Informal Design guidelines for Relational Schema

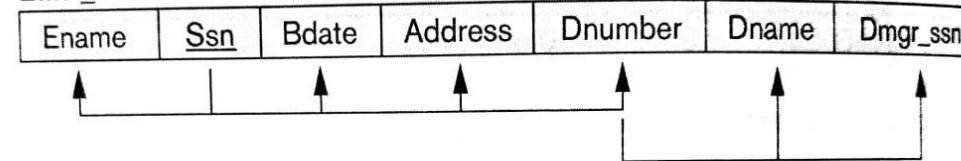
- ▶ Making sure that the semantics of the attributes is clear in the schema
- ▶ Reducing the redundant information in tuple
- ▶ Reducing the NULL values in the tuples
- ▶ Disallowing the possibility of generating spurious tuples

# Imparting clear semantics to attributes in relations

- ▶ The semantics of a relation refers to its meaning resulting from the interpretation of attribute values in a tuple
- ▶ **Guideline-1:** Do not combine attributes from multiple entity types and relationship types into a single relation
- ▶ Ex. Violating guideline 1

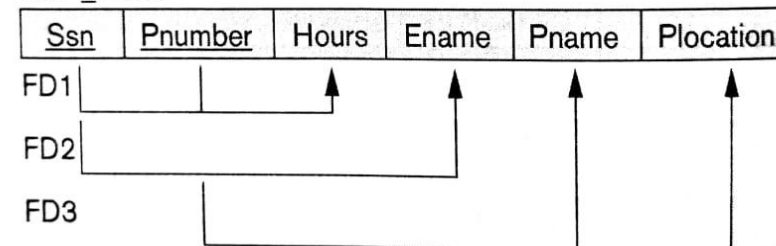
(a)

EMP\_DEPT



(b)

EMP\_PROJ



# Redundant Information in Tuples and Update Anomalies

- ▶ **Insertion Anomalies:** Considering Emp\_Dept relation
  - ▶ Here to insert new employee tuple into the Emp\_Dept relation then, we must either include the values for the department, in which the employee will work for or NULLs if not assigned any department yet
  - ▶ It is also difficult to insert a new department that has no employee. The possible option is to put NULL values in all attributes of employee
- ▶ **Deletion Anomalies**
- ▶ **Modification Anomalies:**
  - ▶ Ex. Modify the manager of a department will force to update all the employee row also who work for that department
- ▶ **Guideline 2:** Design the base relation schema so that no anomalies are present in the relations.

# NULL values in Tuples

- ▶ **Guideline 3:**
- ▶ Avoid placing attributes in a base relation whose values may frequently be null.
- ▶ If NULLs are unavoidable, make sure that they apply in exceptional cases only do not apply to majority of tuples in the relation

## Generation of spurious tuples

- ▶ **Guideline 4:**
  - ▶ Design relational schemas so that they can be joined with equality conditions on attributes that are appropriately related pairs (PK & FK) in a way that guarantees that no spurious tuples are generated

# Codd's Rules

- ▶ Every database has cannot be referred to as a rational database system.
- ▶ If any database has only relational data model, it cannot be a Relational Database System (RDBMS).
- ▶ So, some rules define a database to be the correct RDBMS.
- ▶ These rules were developed by Dr. Edgar F. Codd (E.F. Codd) in 1985.
- ▶ Codd presents his 13 rules for a database to test the concept of DBMS against his relational model, and if a database follows the rule, it is called a true relational database (RDBMS).
- ▶ These 13 rules are popular in RDBMS, known as Codd's 12 rules

- ▶ Rule 0: The Foundation Rule
  - ▶ The database must be in relational form. So that the system can handle the database through its relational capabilities.
- ▶ Rule 1: Information Rule
  - ▶ A database contains various information, and this information must be stored in each cell of a table in the form of rows and columns.
- ▶ Rule 2: Guaranteed Access Rule
  - ▶ Every single or precise data (atomic value) may be accessed logically from a relational database using the combination of primary key value, table name, and column name.
- ▶ Rule 3: Systematic Treatment of Null Values
  - ▶ This rule defines the systematic treatment of Null values in database records. The null value has various meanings in the database, like missing the data, no value in a cell, inappropriate information, unknown data and the primary key should not be null.

- ▶ Rule 4: Active/Dynamic Online Catalog based on the relational model
- ▶ It represents the entire logical structure of the descriptive database that must be stored online and is known as a database dictionary. It authorizes users to access the database and implement a similar query language to access the database.
- ▶ Rule 5: The comprehensive data sublanguage rule:
  - ▶ A relational system may support several languages and various modes of terminal use (for example, the fill-in-the-blanks mode). However, there must be at least one language whose statements are expressible, per some well-defined syntax, as character strings and that is comprehensive in supporting all of the following items:
    - ▶ Data definition.
    - ▶ View definition.
    - ▶ Data manipulation (interactive and by program).
    - ▶ Integrity constraints.
    - ▶ Authorization.
    - ▶ Transaction boundaries (begin, commit and rollback).



- ▶ Rule 6: View Updating Rule

- ▶ All views table can be theoretically updated and must be practically updated by the database systems.

- ▶ Rule 7: Relational Level Operation (High-Level Insert, Update and delete) Rule

- ▶ A database system should follow high-level relational operations such as insert, update, and delete in each level or a single row. It also supports union, intersection and minus operation in the database system.

- ▶ Rule 8: Physical Data Independence Rule

- ▶ All stored data in a database or an application must be physically independent to access the database. Each data should not depend on other data or an application. If data is updated or the physical structure of the database is changed, it will not show any effect on external applications that are accessing the data from the database.

- ▶ Rule 9: Logical Data Independence Rule

- ▶ It is similar to physical data independence. It means, if any changes occurred to the logical level (table structures), it should not affect the user's view (application). For example, suppose a table either split into two tables, or two table joins to create a single table, these changes should not be impacted on the user view application.

- ▶ Rule 10: Integrity Independence Rule

- ▶ A database must maintain integrity independence when inserting data into table's cells using the SQL query language. All entered values should not be changed or rely on any external factor or application to maintain integrity. It is also helpful in making the database-independent for each front-end application.

▶ Rule 11: Distribution Independence Rule

- ▶ The distribution independence rule represents a database that must work properly, even if it is stored in different locations and used by different end-users. Suppose a user accesses the database through an application; in that case, they should not be aware that another user uses particular data, and the data they always get is only located on one site. The end users can access the database, and these access data should be independent for every user to perform the SQL queries.

▶ Rule 12: Non Subversion Rule

- ▶ The non-submersion rule defines RDBMS as a SQL language to store and manipulate the data in the database. If a system has a low-level or separate language other than SQL to access the database system, it should not subvert or bypass integrity to transform data.

# Functional dependency

- ▶ A functional dependency is a constraint that specifies the relationship between two sets of attributes where one set can accurately determine the value of other sets.
- ▶ A functional dependency is denoted by an arrow “ $\rightarrow$ ”. The functional dependency  $X \rightarrow Y$ , where  $X$  is a set of attributes that is capable of determining the value of  $Y$ .
- ▶ The attribute set on the left side of the arrow,  $X$  is called Determinant, while on the right side,  $Y$  is called the Dependent.
- ▶ Functional dependencies are used to mathematically express relations among database entities and are very important to understand advanced concepts in Relational Database System

- From the table we can conclude some valid functional dependencies:

$\text{roll\_no} \rightarrow \{ \text{name}, \text{dept\_name}, \text{dept\_building} \}$ ,  
Here, roll\_no can determine values of fields name, dept\_name and dept\_building,

$\text{roll\_no} \rightarrow \text{dept\_name}$  , Since, roll\_no can determine whole set of {name, dept\_name, dept\_building}, it can determine its subset dept\_name also.

$\text{dept\_name} \rightarrow \text{dept\_building}$  , Dept\_name can identify the dept\_building accurately, since departments with different dept\_name will also have a different dept\_building

More valid functional dependencies:  $\text{roll\_no} \rightarrow \text{name}$ ,  
 $\{ \text{roll\_no}, \text{name} \} \twoheadrightarrow \{ \text{dept\_name}, \text{dept\_building} \}$ , etc.

| roll_no | name | dept_name | dept_building |
|---------|------|-----------|---------------|
| 42      | abc  | CO        | A4            |
| 43      | pqr  | IT        | A3            |
| 44      | xyz  | CO        | A4            |
| 45      | xyz  | IT        | A3            |
| 46      | mno  | EC        | B2            |
| 47      | jkl  | ME        | B2            |

# Armstrong's axioms/properties of functional dependencies:

- ▶ Reflexivity: If  $Y$  is a subset of  $X$ , then  $X \rightarrow Y$  holds by reflexivity rule
  - ▶ For example,  $\{\text{roll\_no}, \text{name}\} \rightarrow \text{name}$  is valid.
- ▶ Augmentation: If  $X \rightarrow Y$  is a valid dependency, then  $XZ \rightarrow YZ$  is also valid by the augmentation rule.
  - ▶ For example, If  $\{\text{roll\_no}, \text{name}\} \rightarrow \text{dept\_building}$  is valid, hence  $\{\text{roll\_no}, \text{name}, \text{dept\_name}\} \rightarrow \{\text{dept\_building}, \text{dept\_name}\}$  is also valid.→
- ▶ Transitivity: If  $X \rightarrow Y$  and  $Y \rightarrow Z$  are both valid dependencies, then  $X \rightarrow Z$  is also valid by the Transitivity rule.
  - ▶ For example,  $\text{roll\_no} \rightarrow \text{dept\_name}$  &  $\text{dept\_name} \rightarrow \text{dept\_building}$ , then  $\text{roll\_no} \rightarrow \text{dept\_building}$  is also valid.

# Types of Functional dependencies

- ▶ Trivial functional dependency
- ▶ Non-Trivial functional dependency
- ▶ Multivalued functional dependency
- ▶ Transitive functional dependency

# 1. Trivial Functional Dependency

- ▶ In Trivial Functional Dependency, a dependent is always a subset of the determinant.
- ▶ i.e. If  $X \rightarrow Y$  and  $Y$  is the subset of  $X$ , then it is called trivial functional dependency
- ▶ Here,  $\{\text{roll\_no}, \text{name}\} \rightarrow \text{name}$  is a trivial functional dependency, since the dependent name is a subset of determinant set  $\{\text{roll\_no}, \text{name}\}$
- ▶ Similarly,  $\text{roll\_no} \rightarrow \text{roll\_no}$  is also an example of trivial functional dependency.

| roll_no | name | age |
|---------|------|-----|
| 42      | abc  | 17  |
| 43      | pqr  | 18  |
| 44      | xyz  | 18  |



## 2. Non-trivial Functional Dependency

- ▶ In Non-trivial functional dependency, the dependent is strictly not a subset of the determinant.
- ▶ i.e. If  $X \rightarrow Y$  and  $Y$  is not a subset of  $X$ , then it is called Non-trivial functional dependency.
- ▶ Here,  $\text{roll\_no} \rightarrow \text{name}$  is a non-trivial functional dependency, since the dependent name is not a subset of determinant roll\_no
- ▶ Similarly,  $\{\text{roll\_no}, \text{name}\} \rightarrow \text{age}$  is also a non-trivial functional dependency, since age is not a subset of  $\{\text{roll\_no}, \text{name}\}$

| roll_no | name | age |
|---------|------|-----|
| 42      | abc  | 17  |
| 43      | pqr  | 18  |
| 44      | xyz  | 18  |

### 3. Multivalued Functional Dependency

- ▶ In Multivalued functional dependency, entities of the dependent set are not dependent on each other.
- ▶ i.e. If  $a \twoheadrightarrow \{b, c\}$  and there exists no functional dependency between  $b$  and  $c$ , then it is called a multivalued functional dependency.
- ▶ It is represented by  $\twoheadrightarrow$  Double Arrow
- ▶ Ex. In the table DNumber  $\twoheadrightarrow$  Dname, Dnumber  $\twoheadrightarrow$  Dlocation But there is no dependency between Dname and Dlocation, So Dnumber  $\twoheadrightarrow$  DLocation

**DEPARTMENT**

| Dname          | <u>Dnumber</u> | Dmgr_ssn  | <u>Dlocation</u> |
|----------------|----------------|-----------|------------------|
| Research       | 5              | 333445555 | Bellaire         |
| Research       | 5              | 333445555 | Sugarland        |
| Research       | 5              | 333445555 | Houston          |
| Administration | 4              | 987654321 | Stafford         |
| Headquarters   | 1              | 888665555 | Houston          |

## 4. Transitive Functional Dependency

- ▶ In transitive functional dependency, dependent is indirectly dependent on determinant.
- ▶ i.e. If  $a \rightarrow b$  &  $b \rightarrow c$ , then according to axiom of transitivity,  $a \rightarrow c$ . This is a transitive functional dependency
- ▶ Here,  $\text{enrol\_no} \rightarrow \text{dept}$  and  $\text{dept} \rightarrow \text{building\_no}$
- ▶ Hence, according to the axiom of transitivity,  $\text{enrol\_no} \rightarrow \text{building\_no}$  is a valid functional dependency.
- ▶ This is an indirect functional dependency, hence called Transitive functional dependency.

| enrol_no | name | dept | building_no |
|----------|------|------|-------------|
| 42       | abc  | CO   | 4           |
| 43       | pqr  | EC   | 2           |
| 44       | xyz  | IT   | 1           |
| 45       | abc  | EC   | 2           |

# Transitive FD example

| Roll_Number | Pin_Code | City_Name |
|-------------|----------|-----------|
| O11         | 450331   | Burhanpur |
| O12         | 450001   | Khandwa   |
| O13         | 456001   | Ujjain    |
| O14         | 452020   | Indore    |

From above table,

Roll\_Number --> Pin\_Code and Pin\_Code --> City\_Name hold.

Then Roll\_Number --> City\_Name is a transitive FD.

# Normalization

- ▶ It is the process of decomposing unsatisfactory "bad" relations by breaking up their attributes into smaller relations
- ▶ Normalization is carried out in practice so that the resulting designs are of high quality and meet the desirable properties
- ▶ Normal form:
  - ▶ Condition using keys and FDs of a relation to certify whether a relation schema is in a particular normal form

# Definitions of Keys and Attributes Participating in Keys

Database\_Keys.pptx

- ▶ A **super key** of a relation schema  $R = \{A_1, A_2, \dots, A_n\}$  is a set of attributes  $S$  subset-of  $R$  with the property that no two tuples  $t_1$  and  $t_2$  in any legal relation state  $r$  of  $R$  will have  $t_1[S] = t_2[S]$
- ▶ A **Candidate key**  $K$  is a super key with the additional property that removal of any attribute from  $K$  will cause  $K$  not to be a super key any more.

OR

- ▶ Minimal Super Key with unique and not null values can be a candidate key
- ▶ One of the candidate keys is arbitrarily designated to be the **primary key**, and the others are called secondary Keys / Alternate Keys
- ▶ A **Prime attribute** must be a member of some candidate key
- ▶ A **Nonprime** attribute is not a prime attribute— that is, it is not a member of any candidate key

# Super Key

- ▶ Definition: Set of one or more attributes that can uniquely identify a tuple.
- ▶ Can include extra attributes not necessary for uniqueness.
- ▶ Example: STUD\_NO, (STUD\_NO, STUD\_NAME)
- ▶ Super key example: (STUD\_NO, PHONE)

| STUD_NO | SNAME  | ADDRESS | PHONE     |
|---------|--------|---------|-----------|
| 1       | Shyam  | Delhi   | 123456789 |
| 2       | Rakesh | Kolkata | 223365796 |
| 3       | Suraj  | Delhi   | 175468965 |

A super key could be a combination of STUD\_NO and PHONE, as this combination uniquely identifies a student.

# Candidate Key

- ▶ Definition: Minimal set of attributes that can uniquely identify a tuple.
- ▶ A minimal super key with no extra attributes.
- ▶ Example: STUD\_NO in STUDENT table
- ▶ Composite key: {STUD\_NO, COURSE\_NO} in STUDENT\_COURSE

**Example:** For the STUDENT table below, STUD\_NO can be a candidate key, as it uniquely identifies each record.

| STUD_NO | SNAME  | ADDRESS | PHONE     |
|---------|--------|---------|-----------|
| 1       | Shyam  | Delhi   | 123456789 |
| 2       | Rakesh | Kolkata | 223365796 |
| 3       | Suraj  | Delhi   | 175468965 |



# Primary Key

- ▶ Chosen candidate key to uniquely identify records.
- ▶ Must be unique and not NULL.
- ▶ Can be single-column or composite.
- ▶ Example: STUD\_NO in STUDENT table

## Example:

*STUDENT table -> Student(STUD\_NO, SNAME, ADDRESS, PHONE) , STUD\_NO is a primary key*

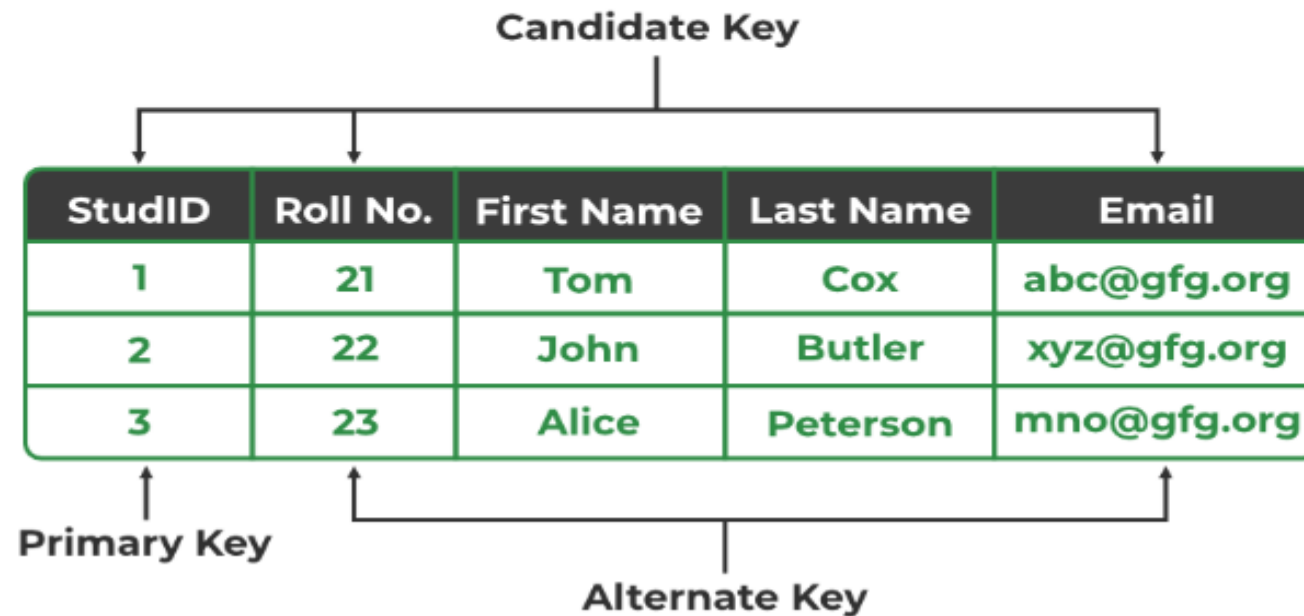
**Table: STUDENT**

| STUD_NO | SNAME  | ADDRESS | PHONE     |
|---------|--------|---------|-----------|
| 1       | Shyam  | Delhi   | 123456789 |
| 2       | Rakesh | Kolkata | 223365796 |
| 3       | Suraj  | Delhi   | 175468965 |

# Alternate Key

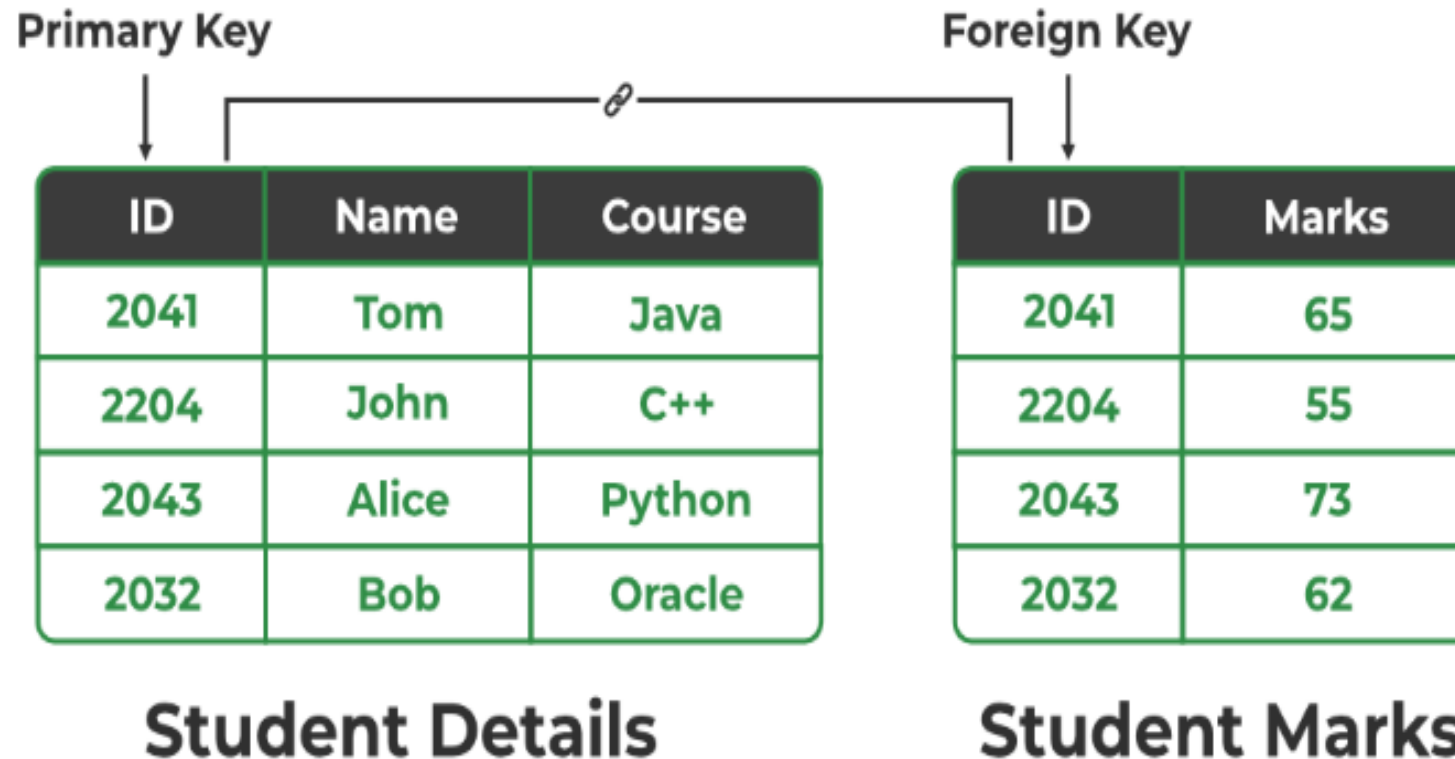
- ▶ Candidate key not chosen as the primary key.
- ▶ Also called secondary key.
- ▶ Example: If STUD\_NO is primary, PHONE is alternate key

**Example:** In the STUDENT table, both STUD\_NO and PHONE are candidate keys. If STUD\_NO is chosen as the primary key, then PHONE would be considered an alternate key.



# Foreign Key

- ▶ Attribute in one table that references primary key of another table.
- ▶ Ensures data integrity and relationship between tables.
- ▶ Example: STUD\_NO in STUDENT\_COURSE references STUDENT.STUD\_NO



# Composite Key

- ▶ Combination of two or more attributes to uniquely identify a row.
- ▶ Can act as primary key if no other key exists.
- ▶ Example: {STUD\_NO, COURSE\_NO} in STUDENT\_COURSE table

**Example:** Consider the STUDENT\_COURSE table

| STUD_NO | TEACHER_NO | COURSE_NO |
|---------|------------|-----------|
| 1       | 005        | C001      |
| 2       | 056        | C005      |

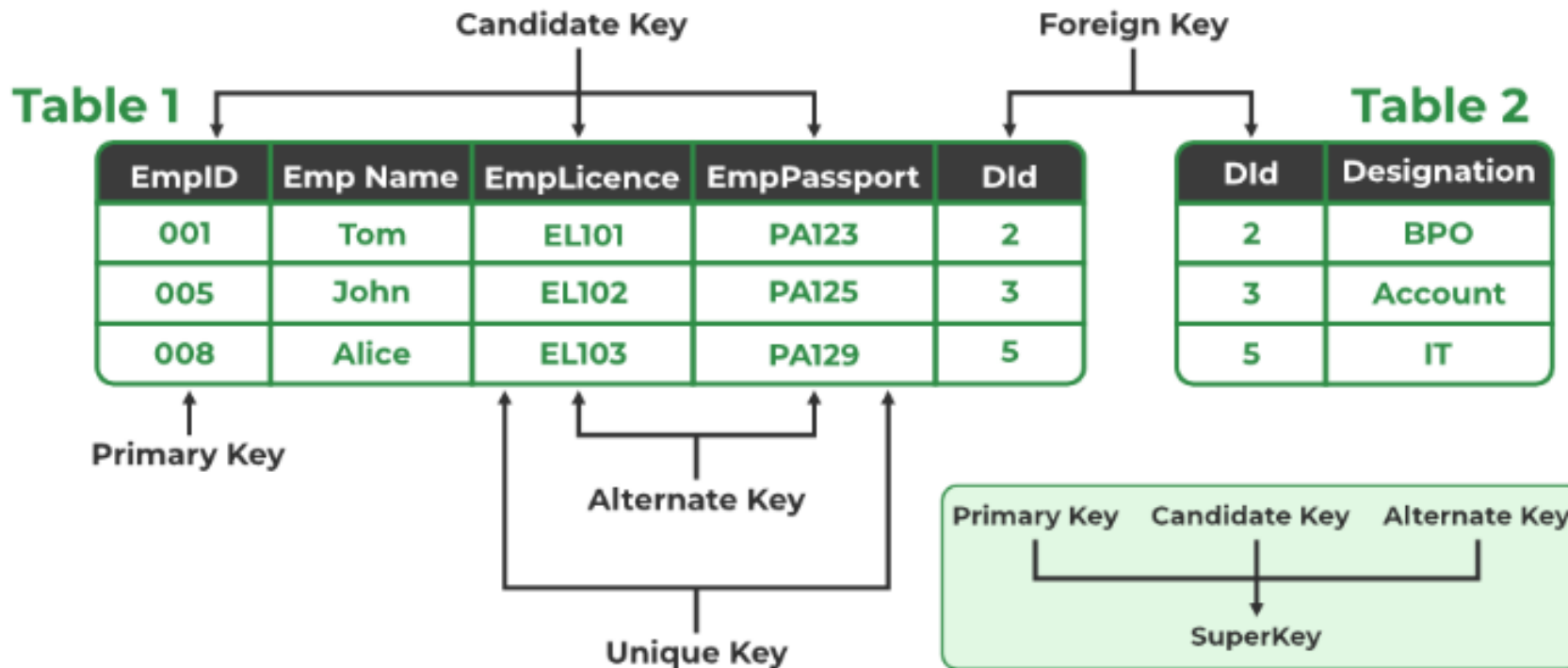
# Summary Table of Keys

- ▶ Super Key: (STUD\_NO, PHONE)
- ▶ Candidate Key: STUD\_NO
- ▶ Primary Key: STUD\_NO
- ▶ Alternate Key: PHONE
- ▶ Foreign Key: STUD\_NO in STUDENT\_COURSE
- ▶ Composite Key: {STUD\_NO, COURSE\_NO}

| STUD_NO (PK) | NAME        | PHONE (Alternate Key) | CITY   |
|--------------|-------------|-----------------------|--------|
| S101         | Riya Sharma | 9876543210            | Delhi  |
| S102         | Aarav Mehta | 9123456789            | Mumbai |
| S103         | Neha Patel  | 9988776655            | Pune   |

| COURSE_NO (PK) | COURSE_NAME |
|----------------|-------------|
| C01            | DBMS        |
| C02            | Java        |
| C03            | Python      |

| STUD_NO (FK) | COURSE_NO (FK) |
|--------------|----------------|
| S101         | C01            |
| S101         | C02            |
| S102         | C01            |
| S103         | C03            |



# Normalization

## First Normal Form(1NF):

- ▶ This rule defines that all the attributes in a relation must have atomic domains. The values in an atomic domain are indivisible units.

▶ Eg.

| Car_model | Color              | Price             |
|-----------|--------------------|-------------------|
| 1         | Red, Grey, Black   | 5lk , 6lk , 4.5lk |
| 2         | Red, Yellow, White | 5.5 lk , 4lk, 6lk |
| 3         | Black, White       | 7lk , 6lk         |
| 4         | White, Grey        | 6lk , 7lk         |

- ▶ Convert into 1<sup>st</sup> NF as:  
{Car\_Model, Color, Price} with multiple records for Car Model

Here in 1<sup>st</sup> NF we don't create any Primary key.

## Second Normal Form(2NF):

- ▶ Every non-prime attribute should be fully functionally dependent on prime key attribute.
- ▶ That is, if  $X \rightarrow A$  holds, then there should not be any proper subset  $Y$  of  $X$ , for which  $Y \rightarrow A$  also holds true.

Eg.

$R \Rightarrow \text{Stud\_Proj} = \{\text{stud\_id}, \text{name}, \text{proj\_id}, \text{proj\_name}\}$

Student:  $\{\text{stud\_id}, \text{name}, \text{proj\_id}, \}$

Project:  $\{\text{proj\_id}, \text{proj\_name}\}$

| stud_id | name        | proj_id | proj_name       |
|---------|-------------|---------|-----------------|
| S101    | Riya Sharma | P01     | Library System  |
| S102    | Aarav Mehta | P02     | Online Shopping |
| S103    | Neha Patel  | P03     | Weather App     |
| S104    | Karan Verma | P01     | Library System  |
| S105    | Priya Desai | P02     | Online Shopping |



# Third Normal Form(3NF):

- ▶ For a relation to be in Third Normal Form, it must be in Second Normal form and the following must satisfy
- ▶ No non-prime attribute is transitively dependent on prime key attribute.
- ▶ For any non-trivial functional dependency,  
 $X \rightarrow A$ , then either
  - ▶  $X$  is a superkey or
  - ▶  $A$  is prime attribute.

Ex.

Stud\_Proj = {stud\_id, name, proj\_id, proj\_name}

Student: {stud\_id, name, proj\_id,}

Project: {proj\_id, proj\_name}

- ▶  $\text{stud\_id} \rightarrow \text{proj\_id}$  ,  $\text{proj\_id} \rightarrow \text{proj\_name}$

## Example of Normalization

| StudentID | StudentName | CourseID | CourseName | CourseFee | InstructorName | InstructorPhone |
|-----------|-------------|----------|------------|-----------|----------------|-----------------|
| 1         | Amit        | C101     | DBMS       | 5000      | Prof. Mehta    | 9876543210      |
| 2         | Rina        | C102     | Python     | 6000      | Prof. Shah     | 9876543222      |
| 3         | Kiran       | C101     | DBMS       | 5000      | Prof. Mehta    | 9876543210      |

### Step 1: Check 1NF

- All attributes contain atomic values. Already in 1NF.

## Step 2: Check 2NF

- Primary Key could be (StudentID, CourseID).
- CourseName, CourseFee, InstructorName, InstructorPhone depend only on CourseID, not on StudentID → **partial dependency**.
- So, split into two tables:

| StudentID | StudentName | CourseID |
|-----------|-------------|----------|
| 1         | Amit        | C101     |
| 2         | Rina        | C102     |
| 3         | Kiran       | C101     |

| CourseID | CourseName | CourseFee | InstructorName | InstructorPhone |
|----------|------------|-----------|----------------|-----------------|
| C101     | DBMS       | 5000      | Prof. Mehta    | 9876543210      |
| C102     | Python     | 6000      | Prof. Shah     | 9876543222      |

### Step 3: Check 3NF

- In COURSE table, InstructorPhone depends on InstructorName, not directly on CourseID.
- This is a transitive dependency.

### Decompose to 3NF

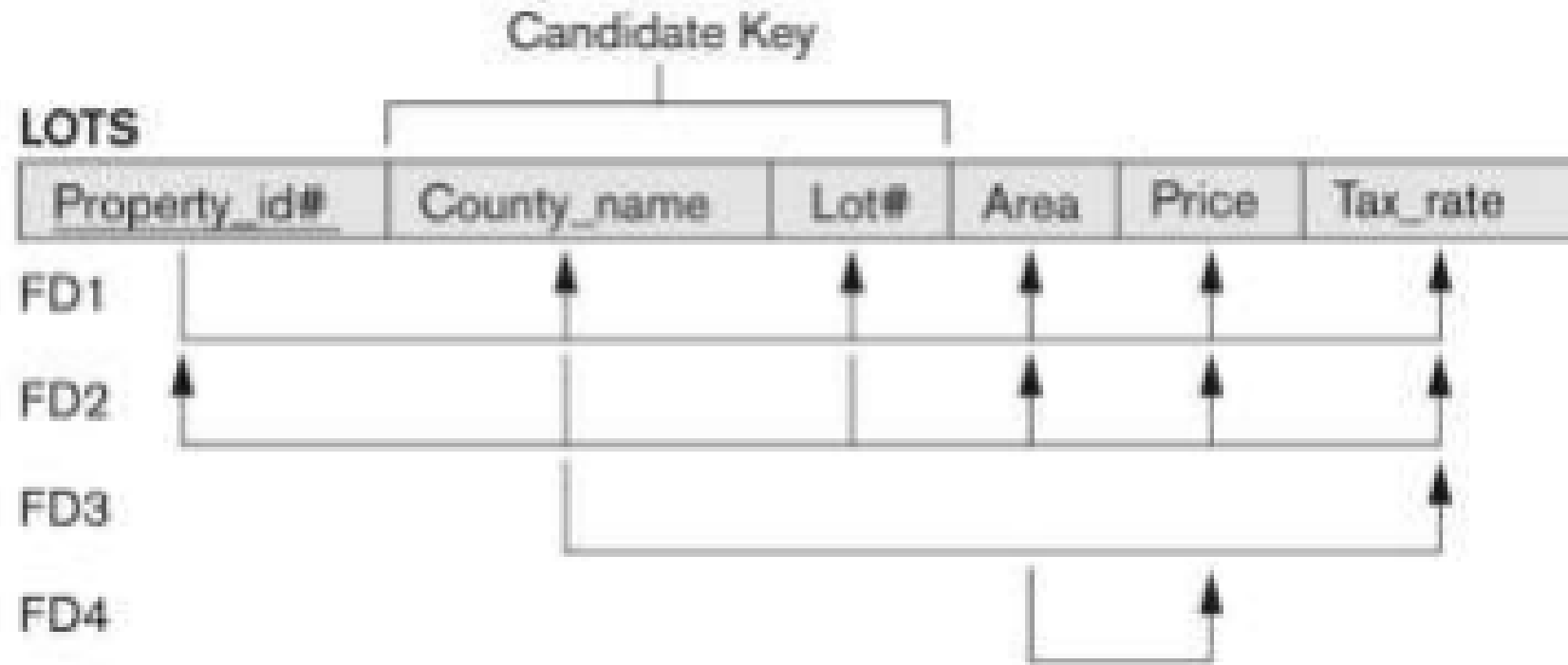
We create a separate INSTRUCTOR Table.

| StudentID | StudentName | CourseID |
|-----------|-------------|----------|
| 1         | Amit        | C101     |
| 2         | Rina        | C102     |
| 3         | Kiran       | C101     |

| CourseID | CourseName | CourseFee | InstructorName |
|----------|------------|-----------|----------------|
| C101     | DBMS       | 5000      | Prof. Mehta    |
| C102     | Python     | 6000      | Prof. Shah     |

| InstructorName | InstructorPhone |
|----------------|-----------------|
| Prof. Mehta    | 9876543210      |
| Prof. Shah     | 9876543222      |

# Example for Normalization



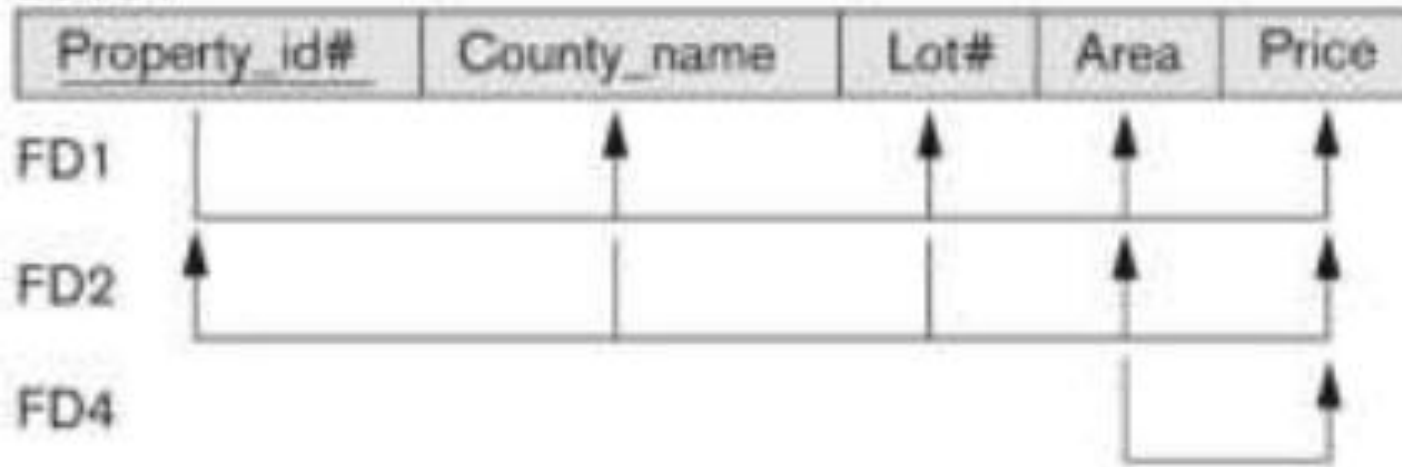
The above relation describes parcels of land for sale in various county of a state.  
Suppose there are 2 candidate keys as:

1. Property\_ID
2. County\_Name and Lot\_No

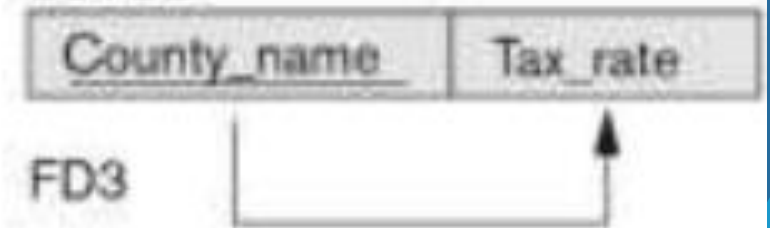
## 2 NF Conversion

- ▶ Remove partial functional dependency
- ▶ County\_Name -> Tax\_Rate

LOTS1



LOTS2



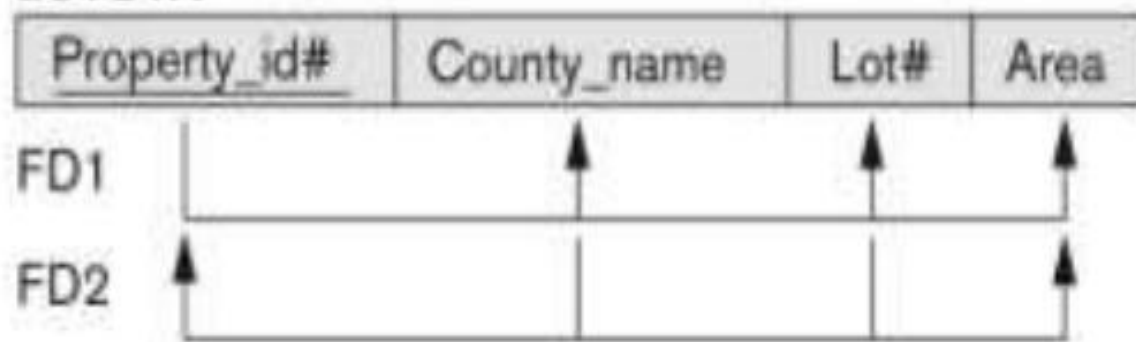
### 3 NF Conversion of relation LOT1

- ▶ Here remove transitive dependency
- ▶ Area  $\rightarrow$  Price and
- ▶ Property\_Id  $\rightarrow$  Area

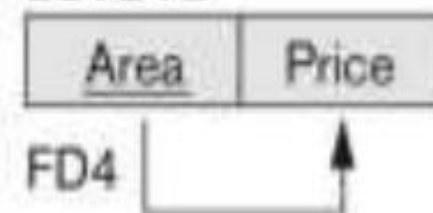
LOTS2



LOTS1A



LOTS1B



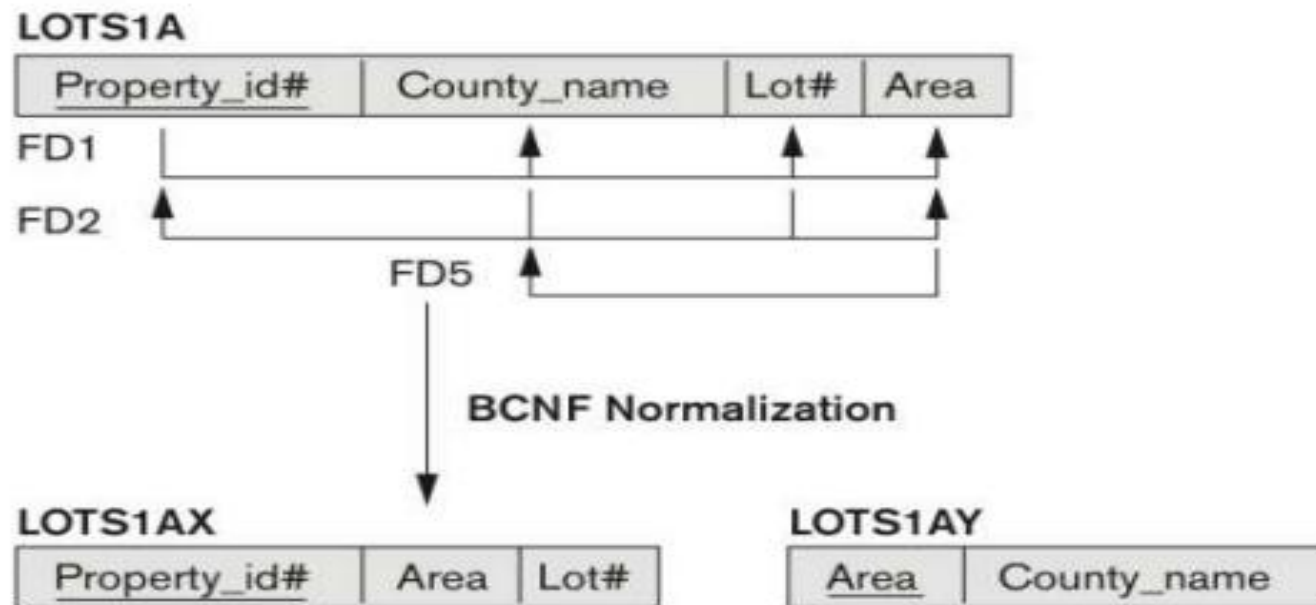
# BCNF – Boyce Codd Normal Form

- ▶ BCNF is the advance version of 3NF. It is stricter than 3NF.
- ▶ A table is in BCNF if every functional dependency  $X \rightarrow Y$ ,
  - ▶ **X is the super key of the table.**
- ▶ For BCNF, the table should be in 3NF, and for every FD, LHS is super key.
- ▶ General definition of 3NF is:
  - ▶ For any FD  $X \rightarrow Y$ :
    - I. X is a super Key of R      OR
    - II. Y is a Prime Attribute of R



# BCNF

- ▶ In the above relation LOTS1A if we assume that the Lot size/ area is fix per county and they are from 2 county only
- ▶ So in that case we can add one more FD i.e. Area -> County\_Name
- ▶ So, according to 3NF this is allowed FD as: Area is not a super key according to rule (i) but as rule (ii) says County\_Name is a prime attribute.
- ▶ But according to BCNF only one condition is allowed for FD is that the **determinant must be a super key**. So, Need to convert the relation LOTS1A into BCNF as shown below:



# BCNF Example

- ▶ Candidate Key:
- ▶ {Student, Course}
- ▶ FDs:
- ▶ {Student, Course}  $\rightarrow$  Instructor
- ▶ Instructor  $\rightarrow$  Course
- ▶ Decomposition Rule:

TEACH

| Student | Course            | Instructor |
|---------|-------------------|------------|
| Narayan | Database          | Mark       |
| Smith   | Database          | Navathe    |
| Smith   | Operating Systems | Ammar      |
| Smith   | Theory            | Schulman   |
| Wallace | Database          | Mark       |
| Wallace | Operating Systems | Ahamad     |
| Wong    | Database          | Omiecinski |
| Zelaya  | Database          | Navathe    |
| Narayan | Operating Systems | Ammar      |

$((R1 \cap R2) \rightarrow (R1 - R2))$  is in  $F^+$ , or  
 $((R1 \cap R2) \rightarrow (R2 - R1))$  is in  $F^+$ .

# Conversion into BCNF

D1: {student, instructor} and {student, course}

D2: {course, instructor} and {course, student}

D3: {instructor, course} and {instructor, student} ✓

- ▶ Out of all above 3 decomposition
- ▶ Only D3 follows the decomposition rule for BCNF
- ▶ So D3 is the only valid decomposition

# Examples for Normalization

R(StudentID, StudentName, CourseID, CourseName, InstructorID, InstructorName, Dept, Grade)

Given functional dependencies (FDs)

FD1 : StudentID  $\rightarrow$  StudentName

FD2 : CourseID  $\rightarrow$  CourseName, InstructorID

FD3 : InstructorID  $\rightarrow$  InstructorName, Dept

FD4 : {StudentID, CourseID}  $\rightarrow$  Grade

# Explanation

- ▶ **Key:** {StudentID, CourseID}

- ▶ After 2NF:

R1: Student (StudentID , StudentName)

R2: Course (CourseID, CourseName, InstructorID, InstructorName, Dept)

R3: Enrollment (StudentID, CourseID, Grade)

- ▶ After 3NF:

R1 and R3 will remain as it is and remove transiti FD from R2

R2a : Instructor(InstructorID PK, InstructorName, Dept)

R2b : Course(CourseID PK, CourseName, InstructorID)

# Examples for Normalization

$R(\text{StuID}, \text{StuName}, \text{Dept}, \text{CourseID}, \text{CourseName}, \text{Instructor}, \text{RoomNo})$

$\text{StuID} \rightarrow \text{StuName}, \text{Dept}$  (Student ID uniquely identifies student info)

$\text{CourseID} \rightarrow \text{CourseName}, \text{Instructor}$  (Each course has a fixed name and instructor)

$\text{Instructor} \rightarrow \text{RoomNo}$  (An instructor is assigned a specific room for lectures)

$(\text{StuID}, \text{CourseID}) \rightarrow (\text{all other attributes})$

(Student enrollment in a course is uniquely identified by  $\text{StuID} + \text{CourseID}$ )

# Decomposition

- ▶  $R = (\text{StuID}, \text{StuName}, \text{Dept}, \text{CourseID}, \text{CourseName}, \text{Instructor}, \text{RoomNo})$  already in 1NF
- ▶ Primary key =  $(\text{StuID}, \text{CourseID})$  (composite).
- ▶ In 2NF, no non-prime attribute should depend on part of composite key.
- ▶ Check partial dependencies:
- ▶  $\text{StuID} \rightarrow \text{StuName}, \text{Dept} \rightarrow$  partial dependency (depends only on StuID).
- ▶  $\text{CourseID} \rightarrow \text{CourseName}, \text{Instructor} \rightarrow$  partial dependency (depends only on CourseID).
- ▶ So 2NF decompose:
- ▶  $R1(\text{StuID}, \text{StuName}, \text{Dept})$
- ▶  $R2(\text{CourseID}, \text{CourseName}, \text{Instructor}, \text{RoomNo})$
- ▶  $R3(\text{StuID}, \text{CourseID})$

# 3NF

Check transitive dependencies:

- In R2:  $\text{CourseID} \rightarrow \text{Instructor}$  and  $\text{Instructor} \rightarrow \text{RoomNo} \rightarrow \text{CourseID} \rightarrow \text{RoomNo}$  (transitive dependency).

Decompose R2:

- **R2a(CourseID, CourseName, Instructor)**
- **R2b(Instructor, RoomNo)**

Now all transitive dependencies are removed.

So after **3NF decomposition**:

1. R1(StuID, StuName, Dept)
2. R2a(CourseID, CourseName, Instructor)
3. R2b(Instructor, RoomNo)
4. R3(StuID, CourseID)



# Normalisation Example - 1

| Dept_Name      | D_No | D_Mgr_SSN | Dept_Location                 |
|----------------|------|-----------|-------------------------------|
| Research       | 5    | 501       | Banglore,<br>Kolkota          |
| Administration | 4    | 401       | Delhi, Mumbai                 |
| Sales          | 1    | 101       | Ahmedabad,<br>Banglore, Delhi |

## Example - 2

| SSN | Proj_No | Emp_Hours | Emp_Name    | Proj_Name    | Proj_Loc  |
|-----|---------|-----------|-------------|--------------|-----------|
| 101 | P1      | 20        | Ankit Shah  | Renovation   | Bopal     |
| 101 | P2      | 50        | Ankit Shah  | Construction | Ghuma     |
| 102 | P2      | 70        | Ankit Shah  | Construction | Bopal     |
| 103 | P2      | 40        | Bhumi Patel | Construction | Sarkhej   |
| 103 | P3      | 30        | Bhumi Patel | Designing    | Maninagar |

## Example - 3

| Property_ID | City | Plot_No | Area | Price |
|-------------|------|---------|------|-------|
|-------------|------|---------|------|-------|

Functional Dependencies are given as below:

FD1:  $\text{Property\_ID} \rightarrow \{\text{City}, \text{Plot\_No}, \text{Area}, \text{Price}\}$

FD2:  $\{\text{City}, \text{Plot\_No}\} \rightarrow \{\text{Property\_ID}, \text{Area}, \text{Price}\}$

FD3:  $\text{Area} \rightarrow \text{Price}$

FD4:  $\text{Area} \rightarrow \text{City}$  (Consider this special condition, FD just for BCNF)

# Multivalued Dependency

- ▶ Multivalued dependency occurs when two attributes in a table are independent of each other but, both depend on a third attribute.
- ▶ A multivalued dependency consists of at least two attributes that are dependent on a third attribute that's why it always requires at least three attributes.

**Example:** Suppose there is a bike manufacturer company which produces two colors(white and black) of each model every year.

| BIKE_MODEL | MANUF_YEAR | COLOR |
|------------|------------|-------|
| M2011      | 2008       | White |
| M2001      | 2008       | Black |
| M3001      | 2013       | White |
| M3001      | 2013       | Black |
| M4006      | 2017       | White |
| M4006      | 2017       | Black |

Here columns COLOR and MANUF\_YEAR are dependent on BIKE\_MODEL and independent of each other. In this case, these two columns can be called as multivalued dependent on BIKE\_MODEL. The representation of these dependencies is shown below:

1. BIKE\_MODEL  $\twoheadrightarrow$  MANUF\_YEAR
2. BIKE\_MODEL  $\twoheadrightarrow$  COLOR

# 4NF

- ▶ Any relation is said to be in the fourth normal form when it satisfies the following conditions:
  - ▶ It must be in Boyce Codd Normal Form (BCNF).
  - ▶ It should have no multi-valued dependency.

## STUDENT

| STU_ID | COURSE    | HOBBY   |
|--------|-----------|---------|
| 21     | Computer  | Dancing |
| 21     | Math      | Singing |
| 34     | Chemistry | Dancing |
| 74     | Biology   | Cricket |
| 59     | Physics   | Hockey  |

# Join Dependency

- ▶ Join decomposition is a further generalization of Multivalued dependencies. If the join of  $R_1$  and  $R_2$  over  $C$  is equal to relation  $R$  then we can say that a join dependency (JD) exists, where  $R_1$  and  $R_2$  are the decomposition  $R_1(A, B, C)$  and  $R_2(C, D)$  of a given relations  $R(A, B, C, D)$ .
- ▶ Alternatively,  $R_1$  and  $R_2$  are a lossless decomposition of  $R$ . A JD  $\bowtie \{R_1, R_2, \dots, R_n\}$  is said to hold over a relation  $R$  if  $R_1, R_2, \dots, R_n$  is a lossless-join decomposition.
- ▶ The  $*(A, B, C, D), (C, D)$  will be a JD of  $R$  if the join of join's attribute is equal to the relation  $R$ .
- ▶ Here,  $*(R_1, R_2, R_3)$  is used to indicate that relation  $R_1, R_2, R_3$  and so on are a JD of  $R$ .

# 5NF

- ▶ A relation is in Fifth Normal Form (5NF), if it is in 4NF, and won't have join dependency.
- ▶ You can also consider that a relation is in 5NF, if the candidate key implies every join dependency in it.

A relation R is in 5NF if and only if every join dependency in R is implied by the candidate keys of R. A relation decomposed into two relations must have loss-less join Property, which ensures that no spurious or extra tuples are generated, when relations are reunited through a natural join.

## Example

The below relation violates the Fifth Normal Form (5NF) of Normalization –

### <Employee>

| EmpName | EmpSkills  | EmpJob (Assigned Work) |
|---------|------------|------------------------|
| David   | Java       | E145                   |
| John    | JavaScript | E146                   |
| Jamie   | jQuery     | E146                   |
| Emma    | Java       | E147                   |

The above relation can be decomposed into the following three tables; therefore, it is not in 5NF –



### <EmployeeSkills>

| EmpName | EmpSkills  |
|---------|------------|
| David   | Java       |
| John    | JavaScript |
| Jamie   | jQuery     |
| Emma    | Java       |

The following is the <EmployeeJob> relation that displays the jobs assigned to each employee –

### <EmployeeJob>

| EmpName | EmpJob |
|---------|--------|
| David   | E145   |
| John    | E146   |
| Jamie   | E146   |
| Emma    | E147   |

Here is the skills that are related to the assigned jobs –

### <JobSkills>

| EmpSkills  | EmpJob |
|------------|--------|
| Java       | E145   |
| JavaScript | E146   |
| jQuery     | E146   |
| Java       | E147   |

## Example 2

A relation R satisfies join dependency if R is equal to the join of  $R_1, R_2, \dots, R_n$  where  $R_i$  is a subset of the set of attributes of R.

### Relation R

| Dept | Subject | Name  |
|------|---------|-------|
| CSE  | C       | Ammu  |
| CSE  | C       | Amar  |
| CSE  | Java    | Amar  |
| IT   | C       | bhanu |

The above relation says that dept offers many elective subjects which are taken by a variety of students. Students have the opinion to choose subjects. Therefore all three fields are needed to represent the information.

The above relation does not show non-trivial MVDs since the attributes subject and name are dependent; they are related to each other (A FD  $\text{subject} \rightarrow \text{name}$  exists). The relation cannot be decomposed in two relations (dept, subject) and (dept, sname).

Therefore the relation can be decomposed into following three relations –

R1(dept, subject)

R2(dept, name) and

R3(subject, name) and it can be shown that decomposition is lossless.

### R1

| Dept | Subject |
|------|---------|
| CSE  | C       |
| CSE  | Java    |
| IT   | C       |

### R2

| Dept | Name  |
|------|-------|
| CSE  | Ammu  |
| CSE  | Amar  |
| IT   | bhanu |

### R3

| Subject | Name  |
|---------|-------|
| C       | Ammu  |
| C       | Amar  |
| Java    | Amar  |
| C       | bhanu |

## EXAMPLE - 3

**Properties** – A relation R is in 5NF if and only if it satisfies following conditions:

1. R should be already in 4NF.
2. It cannot be further non loss decomposed (join dependency)

**Example** – Consider the above schema, with a case as “if a company makes a product and an agent is an agent for that company, then he always sells that product for the company”. Under these circumstances, the ACP table is shown as:

**Table** – ACP

| AGENT | COMPANY | PRODUCT |
|-------|---------|---------|
| A1    | PQR     | Nut     |
| A1    | PQR     | Bolt    |
| A1    | XYZ     | Nut     |
| A1    | XYZ     | Bolt    |
| A2    | PQR     | Nut     |

Table – R1

| AGENT | COMPANY |
|-------|---------|
| A1    | PQR     |
| A1    | XYZ     |
| A2    | PQR     |

Table – R2

| AGENT | PRODUCT |
|-------|---------|
| A1    | Nut     |
| A1    | Bolt    |
| A2    | Nut     |

Table – R3

| COMPANY | PRODUCT |
|---------|---------|
| PQR     | Nut     |
| PQR     | Bolt    |
| XYZ     | Nut     |
| XYZ     | Bolt    |

Result of Natural Join of R1 and R3 over 'Company' and then Natural Join of R13 and R2 over 'Agent'and 'Product' will be table **ACP**.

Hence, in this example, all the redundancies are eliminated, and the decomposition of ACP is a lossless join decomposition. Therefore, the relation is in 5NF as it does not violate the property of lossless join.

## FAQs

### What is join dependency? Give examples.

Whenever we can recreate a table by simply joining various tables where each of these tables consists of a subset of the table's attribute, then this table is known as a Join Dependency.

Thus, it is like a generalization of MVD. We can relate the JD to 5NF. Herein, a relation can be in 5NF only when it's already in the 4NF. Remember that it cannot be further decomposed.

### What is multivalued and join dependency in DBMS?

A multivalued dependency is a case in which a determinant gets associated with some set of values. Whenever isolated, the MVD does not have modification anomalies.

Join dependency or JD is a constraint that is similar to FD (functional dependency) or MVD (multivalued dependency). JD is satisfied only when the concerned relation is a join of a specific number of projections. Thus, such a type of constraint is known as a join dependency.

### What is join dependency in 5NF?

Some relations can be in 5NF whenever it satisfies the 4NF and there exists no join dependency. Thus, a relation can have join dependency in case it can be recreated when we join multiple sub relations, while each of the sub relations contains a subset of the original relation's attributes.

### What is a multivalued dependency, and how do they affect the normalization process?

A multivalued dependency is a case in which a determinant gets associated with some set of values. Whenever isolated, the MVD does not have modification anomalies. Thus, the tables that have these isolated dependencies can be considered to be in the 4NF (fourth normal form).